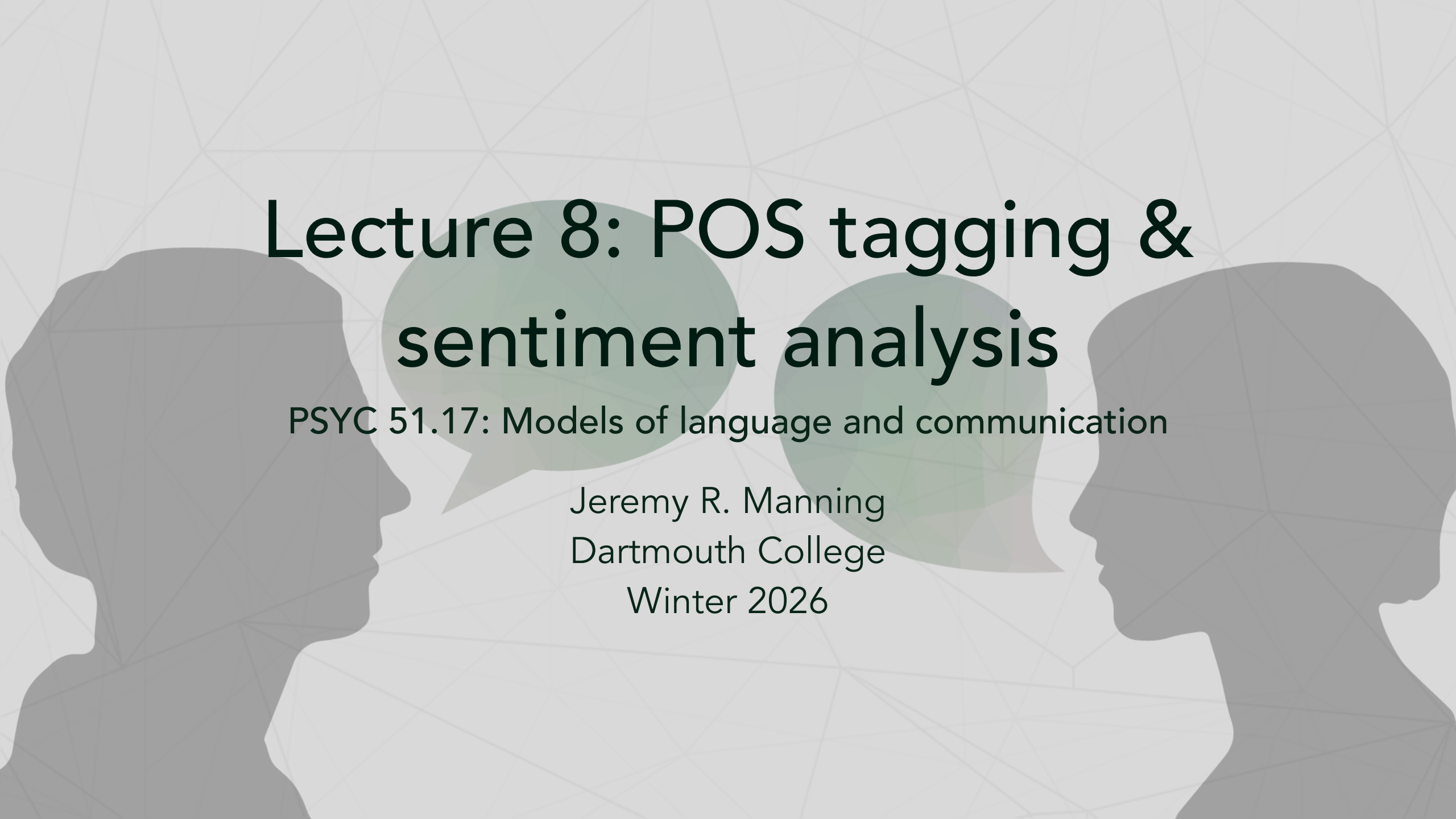


Lecture 8: POS tagging & sentiment analysis

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand part-of-speech (POS) tagging and its applications
2. Explore how neural networks learn grammatical structure
3. Apply sentiment analysis to real-world text
4. Fine-tune pre-trained models for domain-specific tasks
5. Critically evaluate whether models "understand" language

Central questions

- Can statistical patterns capture grammatical knowledge?
- What does it mean for a model to "understand" emotion?

Part-of-speech (POS) tagging

What is POS tagging?

- Assigning grammatical category to each word
- Categories: noun, verb, adjective, adverb, pronoun, preposition, etc.
- A fundamental NLP task

Example

```
The   cat   sat   on   the   mat
DET   NOUN  VERB  ADP  DET   NOUN
```

Why it matters

- Disambiguation: "book" as noun vs. verb
- Syntax parsing and understanding
- Information extraction, machine translation

POS tagsets: universal vs. fine-grained

Universal POS (17 tags):

- ADJ, ADV, ADP, AUX
- CONJ, DET, NOUN, NUM
- PRON, PROPN, VERB, ...

Penn Treebank (45+ tags):

- NN/NNS/NNP/NNPS (nouns)
- VB/VBD/VBG/VBN/VBP/VBZ (verbs)
- Much finer distinctions!

Trade-off

Simplicity vs. linguistic detail

Context matters: ambiguous words

Sentence	Word	POS	Explanation
"I read a **book**"	book	NOUN	Object being read
"Please **book** a table"	book	VERB	Action of reserving
"She runs **fast**"	fast	ADV	Modifies "runs"
"I will **fast** today"	fast	VERB	Action of not eating
"Please **close** the door"	close	VERB	Action
"Stay **close** to me"	close	ADV	Modifies position

Key insight

Context determines POS! Models must look at surrounding words.

spaCy resolves ambiguity using context

Code example

```python
import spacy
nlp = spacy.load("en_core_web_sm")

sentences = [
    "I need to book a flight",       # book = VERB
    "I'm reading a great book",      # book = NOUN
    "The record was broken",         # record = NOUN
    "Please record the meeting",     # record = VERB
]

for sent in sentences:
    doc = nlp(sent)
    for token in doc:
        if token.text.lower() in ["book", "record"]:
            print(f'"{sent}"')
            print(f'  "{token.text}" → {token.pos_} ({spacy.explain(token.pos_)})\n')
```

Context helps to disambiguate parts of speech

Examples: `"to book"` vs `"a book"`. Preceeding words tell us whether `"book"` is a verb or noun.

What methods are used for POS tagging?

Traditional approaches (pre-neural network)

- Rule-based: hand-crafted grammar rules
- Hidden Markov Models (HMMs): probabilistic sequences
- Conditional Random Fields (CRFs): structured prediction

Modern neural network approaches

- Recurrent Neural Networks (RNNs/LSTMs): process sequences
- Transformers (BERT, etc.): bidirectional context
- Fine-tune pre-trained models on POS data

Advantages of neural network approaches

- Learn patterns from data (no hand-crafted rules)
- Capture long-range dependencies
- Handle ambiguity through context
- State-of-the-art accuracy (>97% on English!)

POS tagging is tricky!

Further reading...

Linzen, Dupoux, & Goldberg (2016, *TACL*). Assessing the ability of LSTMs to learn syntax-sensitive dependencies.

Challenge: distractor nouns between subject and verb

The challenge: which noun(s) go with the verb?

- Model must identify "key" (singular) as subject
- Ignore "cabinets" (plural distractor)
- Predict correct verb form "is" (not "are")

Token classification with HuggingFace

```python
from transformers import pipeline
pos_tagger = pipeline("token-classification",
    model="vblagoje/bert-english-uncased-finetuned-pos",
    aggregation_strategy="simple")

sentence = "Apple Inc. is looking at buying a UK startup"
results = pos_tagger(sentence)

for result in results:
    print(f"{result['word']:<15} {result['entity_group']:<8} ({result['score']:.3f})")
# Apple           PROPN    (0.998)
# Inc.            PROPN    (0.995)
# is              AUX      (0.999)
# looking         VERB     (0.997)
```

Further reading

HuggingFace Chapter 7.2: Token Classification

So what? Why is POS tagging important?

Applications

- Syntax parsing
- Named entity recognition
- Information extraction
- Machine translation
- Sentiment analysis (e.g., adjectives)

Linguistic insights

- Study grammatical structure
- Analyze language acquisition
- Explore cross-linguistic patterns
- Understand model "knowledge" of syntax
- Stylometric analysis

Think about it...

Consider how simple rule-based models like ELIZA parse user inputs versus what it means to explicitly model and identify parts of speech. Does POS tagging represent a deeper "understanding" of text? Or is it just another statistical pattern?

Try it out!

Check out the POS tagging demo to explore POS tagging interactively.

Sentiment analysis overview

What is sentiment analysis?

- Identifying emotional tone in text
- Classifying as positive, negative, neutral
- Extracting subjective information

Example

```
"I love this product!"           → Positive
"This is the worst experience."  → Negative
"It's okay, nothing special."    → Neutral
```

Use of POS tagging

Sentiment is often linked to adjectives/adverbs (e.g., "great", "terrible"). POS tagging can help identify sentiment-bearing words. (Caveat: modern models often learn this implicitly!)

Sentiment analysis identifies emotional "tone"

Applications

- Social media monitoring
- Customer feedback analysis
- Market research
- Review analysis
- Political tracking
- Psychological studies (mood/emotion)
- Chatbots and virtual assistants

Granularity levels

- Binary: positive/negative
- Ternary: positive/negative/neutral
- Fine-grained: 1-5 stars
- Continuous: sentiment score
- Aspect-based: per-feature
- Emotion detection: joy, anger, sadness, etc.

Sentiment analysis challenges

Sarcasm and irony

- "Oh great, another meeting" (negative, despite "great")
- "This is the best movie I've ever fallen asleep to" (negative!)

Context-dependent sentiment

- "This movie is sick!" (positive in slang, negative literally)
- "The book was long" (neutral? negative?)

Mixed sentiment

- "Great food but terrible service" (both positive and negative)
- Aspect-based sentiment: food=positive, service=negative

Negation

- "not good" vs. "good"
- "I don't dislike it" (double negative = positive?)

Domain specificity

- "Explosive growth" (positive in business, negative in safety)
- Different domains have different sentiment patterns

Subtlety and ambiguity

- "It's okay, I guess" (neutral or slightly negative?)
- Subjective interpretations vary

Sentiment analysis with HuggingFace

Testing how models handle tricky cases

```python
from transformers import pipeline
sentiment = pipeline("sentiment-analysis")

tricky_cases = [
    ("Oh great, another Monday meeting", "NEGATIVE"),    # Sarcasm
    ("This movie is not bad at all", "POSITIVE"),        # Negation
    ("I don't dislike this product", "POSITIVE"),        # Double negative
    ("Great camera but terrible battery life", "MIXED"), # Mixed sentiment
]

for text, expected in tricky_cases:
    result = sentiment(text)[0]
    print(f"{text}")
    print(f"  Model: {result['label']} ({result['score']:.3f}) | Expected:
{expected}\n")
```

An unsolved problem...

Even modern models still struggle with sarcasm and complex negation!

Further reading

Hugging Face Chapter 1.2: NLP Tasks with Pipeline

There are essentially two types of sentiment analysis approaches

- Lexicon-based methods: predefined word sentiment scores
- Machine learning methods: statistical models (traditional or neural)

Sentiment lexicons: sum up words with predefined sentiment scores

Popular lexicons:

- **AFINN:** -5 to +5 ratings
- **SentiWordNet:** pos/neg/neutral
- **VADER:** Social media focused

Limitations: Ignores context, word order, sarcasm

Example (AFINN):

Word	Score
great	+3
good	+3
hate	-3
terrible	-3
awful	-3

Why use lexicon-based methods?

- Simple and interpretable
- Fast!
- No training data needed; "just works" out of the box

Neural network approaches for sentiment analysis

Neural network advantages

- Learn context-dependent representations
- Capture word order and negation
- Handle sarcasm better (though still imperfect)
- Transfer learning: pre-train on large corpus, fine-tune for sentiment

Typical neural sentiment analysis architecture

Training

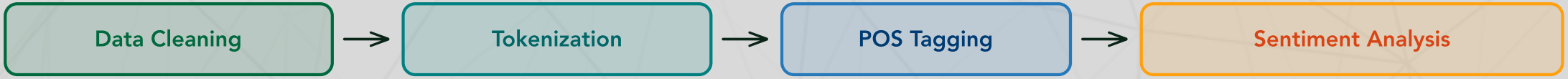
The pre-trained model learns general language patterns, and the classification head is trained specifically for sentiment analysis using labeled sentiment data (IMDb reviews, Amazon reviews, restaurant reviews, etc.).

Discussion: understanding emotion or "just" patterns?

Philosophical questions

1. **Can models "feel" sentiment?**
   - They predict labels, but do they understand emotion?
2. **Is sentiment objective or subjective?**
   - Different annotators may disagree on sentiment
   - How do models handle ambiguity?
3. **Cultural and linguistic variation:**
   - Sentiment expressions vary across cultures
   - Can models capture these nuances?
4. **Ethical considerations:**
   - Automated sentiment analysis in hiring, lending...
   - Risks of bias and discrimination
   - Should we trust model judgments?

Looking back on this week's material



The NLP pipeline (so far!)

- **Data cleaning:** reduces noise by removing unwanted elements
- **Tokenization:** break text into units to enable more efficient processing
- **POS tagging:** reveals grammatical structure
- **Sentiment analysis:** extracts emotional valence and tone

Assignment 2: SPAM email classifier

Apply this week's concepts

- **Data cleaning:** remove HTML tags and special characters, normalize text and formatting
- **Tokenization:** try different tokenizers (word, subword)
- **Features:** extract useful signals (POS patterns, sentiment, keywords, etc.)
- **Classification:** train a model to identify spam vs. legitimate emails

Think about

- Intuitively, what makes spam different from legitimate emails? Can you codify these differences?
- How does preprocessing affect accuracy?
- Can you create compute new features that improve performance?
- Can you leverage POS patterns? (e.g., does spam tend to have more imperative verbs?)
- Which model architectures work best for this task?

Link

Assignment 2: SPAM classifier

Key takeaways

- POS tagging assigns grammatical categories to words using context
- Neural networks (RNNs, Transformers) excel at POS tagging
- Sentiment analysis identifies emotional tone in text
- Lexicon-based and neural network methods have different strengths
- Challenges include sarcasm, negation, and context-dependence
- Critical thinking about model "understanding" is essential

Looking ahead: Weeks 3-4

Next topics

- Dimensionality reduction (PCA, UMAP)
- Word embeddings (Word2Vec, GloVe)
- Distributional semantics

Central question

"You shall know a word by the company it keeps"

How can we represent word *meaning* computationally?

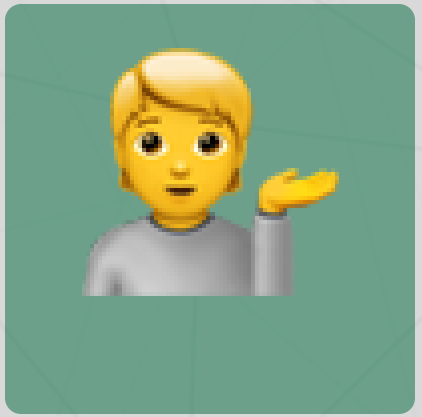
Prepare by

- Finishing Assignment 1 (due Monday!)
- Starting Assignment 2
- Thinking about what "meaning" is. How might you define it?

Questions? Want to chat more?

Email me

Join our Discord



Come to office hours

Congratulations!

Week 2 complete! Have a great weekend and see you next **Wednesday**! 🎉